

Guía: migración de base de datos local a remota (Supabase) y despliegue gratuito (Render + Vercel)

Este documento describe el proceso seguido en el TFG **La Parada** (Django + Angular) y sirve como plantilla para otros proyectos. Incluye el uso de **planes gratuitos** y una sección sobre **opciones de pago**.

1. Conceptos previos

1.1. Qué tenías en local

- **Backend:** Django con **SQLite** (`db.sqlite3`): un solo archivo en disco, ideal para desarrollo.
- **Frontend:** Angular en otro directorio, llamando al API por HTTP (por ejemplo `http://127.0.0.1:8000/api`).

1.2. Qué significa “base de datos en remoto”

Los datos y el esquema (tablas) viven en un **servidor en Internet** (en nuestro caso **PostgreSQL** gestionado por **Supabase**). Tu aplicación Django deja de usar el archivo SQLite y usa una **cadena de conexión** (`DATABASE_URL`) hacia ese servidor.

1.3. Por qué Supabase y no solo “Postgres en Render”

Supabase ofrece PostgreSQL administrado, backups, panel SQL, pooler de conexiones y documentación clara. Render también puede ofrecer Postgres de pago; la idea es la misma: una **URI de conexión** que Django entienda.

2. Migración de datos: de SQLite (local) a PostgreSQL (Supabase)

2.1. Esquema vs datos

- **migrate:** crea o actualiza **tablas** (estructura) en PostgreSQL según los modelos Django.
- **Datos:** los registros que tenías en SQLite hay que **exportarlos** e **importarlos** si quieres conservarlos.

2.2. Preparar Django para PostgreSQL

En el proyecto Django:

1. Añadir dependencias típicas: `psycopg` (driver PostgreSQL), `dj-database-url` (interpretar `DATABASE_URL`), etc. (en `requirements.txt`).
2. En `settings.py`, configurar `DATABASES` para que, si existe la variable de entorno `DATABASE_URL`, use Postgres; si no, siga usando SQLite para desarrollo sin Supabase.

2.3. Obtener la cadena de conexión en Supabase

1. Crea un proyecto en `supabase.com`.
2. **Project Settings** → **Database**.
3. Copia la **Connection string** en modo adecuado:

- **Directa** (db.<ref>.supabase.co:5432): en muchas redes solo expone **IPv6**; en Windows a veces falla el DNS o la conectividad (getaddrinfo failed).
 - **Session pooler (recomendada en IPv4)**: usuario postgres.<REF_PROYECTO>, host aws-0-<REGION>.pooler.supabase.com, puerto 5432. La **región** debe coincidir con la de tu proyecto (por ejemplo eu-west-1).
4. Sustituye [YOUR-PASSWORD] por la contraseña de la base de datos. Si la contraseña tiene caracteres especiales (@, #, %, etc.), deben ir **codificados en URL** (por ejemplo @ → %40).

2.4. Variable de entorno en local (.env)

En la carpeta del backend, archivo .env (no subir a Git):

```
DATABASE_URL=postgresql://postgres.TU_REF:TU_PASSWORD@aws-0-TU_REGION.pooler.supabase.com:5432/postgres
```

Otros valores habituales: SECRET_KEY, DEBUG=True en local, ALLOWED_HOST, CSRF_TRUSTED_ORIGINS para el puerto del Angular (http://localhost:4200).

2.5. Aplicar el esquema en Supabase

Desde la carpeta del backend, con el entorno virtual o Python adecuado:

```
pip install -r requirements.txt
python manage.py migrate
```

Eso crea las tablas en PostgreSQL vacías (salvo tablas internas mínimas de Django).

2.6. Copiar datos desde SQLite (opcional)

Requisito: tener el archivo db.sqlite3 con los datos antiguos.

1. **Exportar** sin usar DATABASE_URL (para que Django lea SQLite): por ejemplo comentar temporalmente DATABASE_URL en .env o usar un script que quite esa línea un momento.
2. Ejecutar:

```
python manage.py dumpdata --natural-foreign --natural-primary --indent 2 -e contenttypes -e auth.Permission -e sessions > datos_export.json
```

3. Volver a poner DATABASE_URL hacia Supabase.

4. Importar:

```
python manage.py loaddata datos_export.json
```

Si hay errores de unicidad o orden, puede hacer falta excluir apps concretas del dumpdata o importar en varios pasos. Los **archivos binarios** (imágenes en ImageField) no van dentro del JSON: las rutas en BD apuntan a ficheros; hay que **copiar la carpeta media/** al servidor o usar almacenamiento en la nube (ver sección de imágenes en despliegue).

3. Git y GitHub: cómo se “comunica” el repositorio

3.1. Qué es Git

Git es control de versiones en tu PC: registra cambios (commits), ramas y historial.

3.2. Qué es GitHub

GitHub es un servidor en la nube donde guardas una **copia** del repositorio. **Render** y **Vercel** se conectan a GitHub, **clonan** el repo en cada despliegue y ejecutan los comandos de build que configures.

Flujo resumido:

1. Tú editas código en local → `git add` → `git commit`.
2. `git push` sube los commits a GitHub.
3. Render/Vercel detectan el push (o puedes lanzar deploy manual) → vuelven a construir y publicar la nueva versión.

3.3. Comandos mínimos

```
cd ruta/al/proyecto
git init                    # solo la primera vez
git branch -M main
git remote add origin https://github.com/USUARIO/REPO.git
git add .
git commit -m "Descripción del cambio"
git push -u origin main
```

3.4. Qué no subir nunca

- Archivos **.env** (secretos, contraseñas de BD).
- **db.sqlite3** si contiene datos sensibles de prueba (opcional según política).
- **node_modules/**, entornos virtuales grandes, etc. (listados en **.gitignore**).

El archivo **.gitignore** evita que `git add .` incluya esos paths. Si alguna vez subiste **.env** por error, quítalo del historial con `git rm --cached` y añade la regla al **.gitignore**.

4. Despliegue del backend en Render

4.1. Qué es Render

Render ejecuta tu backend como un **Web Service** (proceso HTTP persistente). Para Django se usa **Gunicorn** apuntando al WSGI del proyecto.

4.2. Plan gratuito vs de pago

Aspecto	Gratis (Free)	De pago (Starter, Standard, ...)
Precio	0 €	Desde unos 7 \$/mes según plan
“Sleep”	El servicio se duerme tras inactividad; la primera petición puede tardar ~50 s o más en despertar	Instancia siempre encendida (sin cold start de ese tipo)
Recursos	RAM/CPU limitados	Más RAM, CPU, opciones de escala
Disco persistente	No (o muy limitado): archivos subidos al disco pueden perderse al redeploy	Posibilidad de discos persistentes , más conexiones, SLA
Dominio	tu-servicio.onrender.com	Igual + dominios personalizados en todos los planes; en free a veces con limitaciones según política actual de Render

Para un TFG o demo, **Free** suele bastar. Para producción real (tienda 24/7, muchos usuarios), conviene **Starter** o superior y valorar **Postgres en Render** o seguir con Supabase.

4.3. Pasos en Render (Web Service)

1. Cuenta en render.com, conectar **GitHub**.
2. **New** → **Web Service**, elegir el **repositorio** y la **rama** (main).
3. **Root Directory**: carpeta del backend si el repo es monorepo (ej. LPBackend). Si el Django está en la raíz del repo, déjalo vacío.
4. **Runtime**: Python; versión acorde a runtime.txt o configuración del panel.
5. **Build command (ejemplo)**:

```
pip install -r requirements.txt && python manage.py collectstatic --noinput
```

6. **Start command (ejemplo)**:

```
gunicorn NombreProyecto.wsgi:application --bind 0.0.0.0:$PORT
```

(NombreProyecto es el paquete donde está wsgi.py.)

7. **Instance type**: elegir **Free** para no pagar.

4.4. Variables de entorno en Render

Configurar al menos:

- DATABASE_URL — URI de Supabase (pooler sesión si aplica).
- DEBUG — False en producción.

- SECRET_KEY — cadena larga y aleatoria.
- ALLOWED_HOST — solo el hostname del servicio, ej. `mi-api.onrender.com` (sin `https://`).
- CSRF_TRUSTED_ORIGINS — URL(s) del front con `https://`, ej. `https://mi-app.vercel.app`.

Detrás de un proxy HTTPS, en Django conviene `SECURE_PROXY_SSL_HEADER` y `USE_X_FORWARDED_HOST` para que las URLs absolutas (imágenes, redirects) usen **https** correctamente.

4.5. Archivos estáticos y media

- **Static (admin, CSS de Django):** `collectstatic` + **WhiteNoise** (o CDN).
- **Media (subidas de usuarios):** en Free, el disco es **efímero**; lo robusto es **S3 / Supabase Storage**. Para una demo se puede servir `/media/` con cuidado y subir las imágenes del catálogo en el repo o en un volumen de pago.

5. Despliegue del frontend en Vercel

5.1. Qué es Vercel

Vercel sirve el **build estático** del Angular (HTML, JS, CSS). El navegador del usuario descarga la app desde el dominio de Vercel y esta llama al API en Render.

5.2. Plan gratuito vs de pago

Aspecto	Hobby / Free	Pro (de pago)
Precio	0 € en límites del plan hobby	Cuota mensual según plan
Equipo / SSO	Limitado	Más asientos, controles empresariales
Análisis / edge	Básico	Más funciones, límites mayores
Dominios	Dominio <code>*.vercel.app</code> gratis; dominio propio suele estar en free con configuración DNS	Mismas capacidades con más proyectos, límites y soporte

5.3. Pasos en Vercel

1. `vercel.com` → importar proyecto desde **GitHub**.
2. **Root Directory:** carpeta del Angular (ej. `LPAngular`), no la raíz del monorepo si ahí solo está el backend mezclado.
3. **Build command:** por ejemplo `npm run build` o el script que inyecte la URL del API antes del build (`NG_API_URL`).
4. **Output directory:** en Angular reciente suele ser `dist/NombreProyecto/browser`.
5. Variable de entorno **NG_API_URL** (o la que use tu script) con el valor **`https://TU-API.onrender.com/api`** (incluyendo `/api` si tu API está montada así).

Tras el deploy, la web estará en algo como `https://tu-app.vercel.app`.

5.4. Coherencia con el backend

Actualiza en Render **CSRF_TRUSTED_ORIGINS** con la URL exacta del front en Vercel (`https://...vercel.app`). Así el navegador puede usar cookies/sesión o cabeceras sin bloqueos CSRF en rutas que lo requieran.

6. Cambiar o usar un nombre de dominio propio

Quieres que la tienda abra en `https://www.tudominio.com` en lugar de solo `*.vercel.app`.

6.1. Dónde comprar el dominio

Registradores habituales: **Cloudflare Registrar**, **Namecheap**, **Google Domains** (migrado), **OVH**, etc. Compras `tudominio.com` y gestionas los **DNS** (registros A, CNAME, TXT).

6.2. Frontend (Vercel)

1. En el proyecto Vercel: **Settings** → **Domains**.
2. Añade `www.tudominio.com` (y/o `tudominio.com`).
3. Vercel mostrará instrucciones: normalmente un **CNAME** de `www` hacia `cname.vercel-dns.com` (o el valor que indique la UI actual).
4. Espera la propagación DNS (minutos a horas).
5. Actualiza en el backend **CSRF_TRUSTED_ORIGINS** para incluir `https://www.tudominio.com` (y `https://tudominio.com` si usas apex).
6. En Angular, la URL del API debe seguir siendo la del **backend**; si el API también va en subdominio propio, actualiza `NG_API_URL` / `environment` y redeploy del front.

6.3. Backend (Render)

1. En el servicio web: **Settings** → **Custom Domains**.
2. Añade por ejemplo `api.tudominio.com`.
3. Render indicará un **CNAME** o registro **TXT** de verificación.
4. En tu DNS, crea el registro que pida Render.
5. En Django, **ALLOWED_HOST** debe incluir `api.tudominio.com` (sin `https://`).
6. **CSRF_TRUSTED_ORIGINS** ya incluye el front, no hace falta poner el dominio del API salvo que tengas formularios que lo requieran.

6.4. Resumen DNS típico

Nombre	Tipo	Valor (ejemplo)
<code>www</code>	CNAME	destino que indique Vercel
<code>api</code>	CNAME	destino que indique Render
<code>@</code> (apex)	A o ALIAS	según proveedor; a veces rediriges apex → <code>www</code>

6.5. HTTPS

Tanto Vercel como Render suelen emitir **certificados TLS automáticamente** para los dominios que añades. No hace falta comprar certificado aparte para lo básico.

7. Checklist rápido para repetir en otro proyecto

1. Elegir BD remota (Supabase, RDS, Neon, Postgres en Render, etc.) y obtener **URI**.
 2. Adaptar `settings.py` y `requirements.txt` en Django.
 3. `migrate` en remoto; opcional `dumpdata` / `loaddata` desde SQLite.
 4. Crear repo Git, `.gitignore`, push a GitHub.
 5. Render: Web Service, root dir, build/start, env vars, plan Free o de pago.
 6. Vercel: root del front, build, output dir, `NG_API_URL` al API.
 7. Probar; ajustar CORS/CSRF/ALLOWED_HOST.
 8. (Opcional) Dominio propio en DNS + Vercel + Render + variables actualizadas.
-

8. Referencias útiles

- Supabase — conectar a Postgres: <https://supabase.com/docs/guides/database/connecting-to-postgres>
 - Render — despliegue Python: documentación en render.com/docs
 - Vercel — Angular: <https://vercel.com/docs>
 - Django — despliegue: <https://docs.djangoproject.com/en/stable/howto/deployment/>
-

9. Información adicional para futuros despliegues

Esta sección recoge buenas prácticas y “trucos” que aplican a **cualquier proyecto** similar (backend + front + BD remota), no solo al TFG.

9.1. Ramas, pull requests y entornos

- Trabaja en **ramas** (feature/..., fix/...) y fusiona en main cuando esté estable. Así reduces romper producción.
- **Vercel** suele crear una **Preview URL** por cada PR: sirve para probar el front contra el API de **staging** o de producción sin tocar la web principal.
- En **Render**, el servicio de producción suele apuntar solo a main. Puedes crear un **segundo Web Service** (rama develop, plan free aparte) como **staging** con otra `DATABASE_URL` si quieres no mezclar datos.

9.2. Fijar versiones de Python y Node

- En **Render**, usa `runtime.txt` en la raíz del backend (ej. 3.13.0) para que el despliegue no cambie de versión de Python sin aviso.
- En el front, en `package.json` puedes usar el campo **"engines"** (`"node": ">=20"`) para que Vercel/CI avise si la versión no coincide.
- Anota en el README del repo qué versiones usaste; en seis meses lo agradecerás.

9.3. Variables de entorno por “entorno”

- **Vercel**: puedes definir la misma variable (`NG_API_URL`) con valores distintos para **Production**, **Preview** y **Development**. Así las previews apuntan a un API de pruebas.
- **Render**: cada servicio tiene su propio bloque de variables; duplica servicios si necesitas prod/staging con URLs distintas.

9.4. Monorepo (backend + front en un solo repo)

- Cada plataforma solo ve **una carpeta raíz** para build: en Render **LPBackend**, en Vercel **LPAngular** (ejemplo).
- El **render.yaml** (Blueprint) puede vivir en la raíz del repo y referenciar rootDir.
- Evita rutas absolutas de tu PC en el código; todo debe ser **relativo al repo** o **variables de entorno**.

9.5. Fallos frecuentes y qué mirar primero

Síntoma	Causa probable	Qué revisar
Build Render: “Root directory does not exist”	La carpeta no está en GitHub o el nombre no coincide	Árbol del repo en GitHub; mayúsculas (LPBackend vs lpbackend en Linux).
getaddrinfo failed al migrar desde Windows	Conexión directa Supabase solo IPv6	Usar Session pooler y región correcta.
DisallowedHost	ALLOWED_HOST no incluye el host de la petición	Hostname del servicio Render o dominio custom sin https://.
CSRF / cookies raras	Origen del front no está en CSRF_TRUSTED_ORIGINS	URL exacta https://... del Vercel o dominio.
API responde pero el front “no conecta”	URL del API mal en build	NG_API_URL en Vercel; barra final /api coherente con Django urls.py.
Imágenes media/ 404	No se sirven en prod o no hay ficheros en disco	DEBUG, flags de media, archivos en repo o Storage en nube.
Primera petición muy lenta	Plan Free de Render en reposo	Normal; segundo plan o “ping” programado (pago/límite).

9.6. “Cold start” en Render (plan gratuito)

Tras un tiempo sin tráfico, la instancia **se apaga**. La primera petición la **enciende** de nuevo (puede tardar decenas de segundos). Para una demo está bien; para un cliente exigente, usa plan de pago o un **cron externo** que haga ping (con moderación y respetando términos de servicio de Render).

9.7. CORS y seguridad al salir a producción

- En desarrollo a veces se usa `CORS_ALLOW_ALL_ORIGINS = True`; en **producción** conviene **restringir** al dominio del front (`https://tu-app.vercel.app`).
- Rota **SECRET_KEY** y contraseñas si alguna vez se filtraron (chat, captura de pantalla, commit accidental).
- Activa **2FA** en GitHub, Render y Vercel.

9.8. Copias de seguridad de la base de datos (Supabase)

- En el panel de Supabase revisa **Backups** y políticas de retención según tu plan.

- Para un volcado manual ocasional puedes usar **pg_dump** contra la URI del pooler o conexión directa (documentación Supabase).
- Antes de migraciones arriesgadas en producción, exporta un **dump** o haz prueba en otro proyecto Supabase.

9.9. Nuevas versiones del código y de la base de datos

- **Código:** `git push` → redeploy automático (si está activado) o botón **Manual Deploy**.
- **Solo esquema BD:** en local `python manage.py makemigrations` → `commit` → en el servidor (o CI) `migrate` tras el deploy.
- **Datos destructivos:** planifica scripts SQL o migraciones de datos en Django; prueba antes en staging.

9.10. CI/CD con GitHub Actions (opcional)

Puedes automatizar tests en cada push:

1. Workflow en `.github/workflows/ci.yml`.
2. Jobs: instalar Python, `pip install`, `python manage.py test`; otro job con Node, `npm ci`, `npm run build`.
3. El deploy sigue siendo Render/Vercel al mergear a main, o puedes usar la **API de deploy** de Vercel/Render con **secrets** en GitHub (`VERCEL_TOKEN`, etc.) si quieres despliegue solo si los tests pasan.

9.11. Otras plataformas (referencia rápida)

Necesidad	Alternativas habituales
API Python / Node	Railway, Fly.io, Google Cloud Run, Azure App Service
Front estático / SPA	Netlify, Cloudflare Pages, GitHub Pages
Solo contenedores	Fly.io, AWS ECS/Fargate con imagen Docker
Base Postgres gestionada	Supabase, Neon, ElephantSQL, RDS (AWS)

Los mismos conceptos (`DATABASE_URL`, build, variables, dominio) se repiten; cambia el panel donde los introduces.

9.12. Docker (visión general)

Empaquetar backend en una **imagen Docker** permite desplegar la misma build en Render, Fly, AWS, etc. Añade un `Dockerfile`, prueba en local con `docker build` / `docker run`, y en el PaaS eliges “deploy from Dockerfile”. Es más trabajo inicial pero **reproducible** entre equipos.

9.13. Checklist el mismo día del despliegue

- ☐ Último `git push` y `commit` con mensaje claro.
- ☐ `.env` **no** está en el repo; secretos solo en paneles.
- ☐ `ALLOWED_HOST` y `CSRF_TRUSTED_ORIGINS` actualizados si cambió la URL.
- ☐ Probar **GET** a una ruta del API y una página del front en incógnito.
- ☐ Revisar **logs** de build y de runtime en Render y Vercel si algo falla.

- ☐ Anotar en un sitio seguro las URLs finales y qué variable va en cada sitio.

9.14. Volver atrás (rollback)

- **Código:** `git revert` del commit problemático → push, o en el panel de Render/Vercel desplegar un **commit anterior** si la UI lo permite.
- **Base de datos:** sin backup no hay rollback limpio; por eso importan dumps y backups en Supabase.

9.15. Límites y “coste oculto” del gratis

- Límites de **horas/mes**, **ancho de banda**, **build minutes** y **tamaño de repo** pueden aplicarse; lee las páginas de precios actuales de Render y Vercel.
- Un proyecto con mucho tráfico o builds largos puede chocar con el techo del plan gratuito; el mensaje de error suele indicarlo.

9.16. Regenerar este documento en PDF (Pandoc + Typst)

En Windows (PowerShell), tras instalar **Pandoc** y **Typst** (por ejemplo con winget):

```
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") + ";"
+ [System.Environment]::GetEnvironmentVariable("Path","User")
cd ruta\al\proyecto
pandoc DOCUMENTACION_MIGRACION_Y_DESPLIEGUE.md --pdf-engine=typst -o
DOCUMENTACION_MIGRACION_Y_DESPLIEGUE.pdf
```

Si no tienes Typst, Pandoc pedirá `pdflatex` (LaTeX); Typst evita instalar MiKTeX para un PDF sencillo a partir de Markdown.

10. Plantilla mínima de “ficha de proyecto” (copiar y rellenar)

Guarda esto por cada aplicación que despliegues:

Nombre del proyecto:

Repositorio GitHub:

Rama de producción:

--- Backend (ej. Render) ---

URL pública API:

Root directory en el PaaS:

Build command:

Start command:

Variables críticas: `DATABASE_URL`, `SECRET_KEY`, `DEBUG`, `ALLOWED_HOST`, `CSRF_TRUSTED_ORIGINS`, ...

--- Frontend (ej. Vercel) ---

URL pública web:

Root directory:

Build command:

Output directory:

Variables (ej. `NG_API_URL`):

--- Base de datos ---

Proveedor (Supabase / ...):

Región / tipo de conexión (directa / pooler):

¿Dónde están los backups?

--- Dominios ---

Dominio www (Vercel):

Dominio API (Render):

Fecha último despliegue correcto:

Notas:

Documento de apoyo al TFG La Parada. PDF recomendado: Pandoc con --pdf-engine=typst (ver sección 9.16).